

CS633 Assignment Group Number 22

Kishan Kumar Mishra (251110044)
Aditya Pushkar (251110004)
Govind Gupta (251110033)
Harsh Goel (251110034)

February 14, 2026

1 Code Description

1.1 Memory Allocation and Random Data Generation

Listing 1: Memory allocation and data generation for all the processes (P)

```
double *data_for_sending_to_D1 = (double*)malloc(M * sizeof(double));
double *data_for_sending_to_D2 = (double*)malloc(M * sizeof(double));
double *data_received = (double*)malloc(M*sizeof(double));
double max_of_D1 = DBL_MIN, max_of_D2 = DBL_MIN;
int valid_sender = 0;

for(int i = 0; i < M; i++) {
    data_for_sending_to_D1[i] = (double) rand() * (rank + 1) / 10000.0;
    data_for_sending_to_D2[i] = data_for_sending_to_D1[i];
}
```

Explanation:

- We allocate three buffers of type `double*`:
 - * `data_for_sending_to_D1`: For sending data to ranks at distance D_1 ahead.
 - * `data_for_sending_to_D2`: For sending data to ranks at distance D_2 ahead.
 - * `data_received`: For receiving data from ranks at distances D_1 and D_2 (both ahead and behind).
- `max_of_D1` and `max_of_D2` store the maximum data element received from rank D1 distance ahead and rank D2 distance ahead, respectively.
- `valid_sender` is set to 0 initially, which will get updated to 1 later if a rank is a valid sender.
- The for loop prepares the data to be sent to D1 and D2 in the first iteration. The data is generated randomly using the seed(received through the command line). For the 1st iteration, the same data is sent to both D1 and D2.

1.2 Construction of Processes Communication Pattern

1.2.1 Example: Communication Pattern with $P = 16$, $D1 = 2$, $D2 = 4$

Consider a system with 16 processes and distances $D1 = 2$ and $D2 = 4$:

Each process needs to **send data ahead** to processes at distances $D1$ and $D2$ and **receive updated data back** from those same processes after they have processed it.

A process which has a valid destination $D1$ or $D2$ or both is considered a valid sender.

To have maximum parallel communication, we make independent groups of valid senders using the below logic:

1. Start with P_1 : It sends data to P_3 ($D1$) and P_5 ($D2$), and will later receive the updated data from them.
2. Next, pick P_2 : it sends data to P_4 ($D1$) and P_6 ($D2$) and waits for updates back.
3. We cannot pick P_3 since it is having dependency with P_1 . So we skip it and jump to a process which does not have any dependency with the last process(P_2). P_2 has dependency upto $P_2 + D2$ i.e. P_6 . So we jump to P_7 .
4. P_7 and P_8 are processed but P_9 is skipped since it is having dependency with P_7 . We follow the same logic that we followed in step 3 and jump to P_{13} .
5. P_{13} and P_{14} are processed but P_{15} and P_{16} are skipped due to aforementioned logic. Since we have 16 processes only we need not proceed further. At this point, our first independent block(B_1) is ready. It contains processes $P_1, P_2, P_7, P_8, P_{13}, P_{14}$. These processes can do their send and receive without any interference from any other process in the same .
6. Now we proceed to make our next independent block. We go back to processes which were skipped initially. So we process P_3 and P_4 . P_5 is skipped because of dependency with P_3 . So we jump to $P_4 + D2 + 1$ i.e. P_9 . We process P_9 and P_{10} and then jump to P_{15} . We stop here since P_{15} does not have a valid destination. Our 2nd independent block(B_2) contains P_3, P_4, P_9, P_{10} .
7. Our 3rd independent block(B_3) contains processes P_5, P_6, P_{11}, P_{12} .
8. Since there is dependency between the senders in different independent blocks, all communication in an independent block must finish before any communication happens in the next independent block.

1.2.2 Process Information Data Structure

Listing 2: Structure to store process communication information

```
struct pInfo {  
    bool isSend; // state whether sending or receiving  
    int comm_rank; // rank of process to whom we are sending or receiving from  
    bool isatD1orD2; // true for D1, false for D2  
};
```

Explanation:

We have defined a structured data type `pInfo` that stores key information about sending and receiving actions.

- `isSend` indicates whether the process will send (`true`) or receive (`false`) the data.
- `comm_rank` stores the rank of the target process for communication.
- `isAtD1orD2` specifies whether the communication is at distance D1 (`true`) or D2 (`false`).

1.2.3 Important Data Variables used to Generate the Communication Pattern

Listing 3: Defining required variables for comm. pattern generation

```
int continous_ranks = D2 - D1;
int jump = D2 + 1;

struct pInfo** processes_states_flow = (struct pInfo**)malloc(size * sizeof(struct
    pInfo*));

int* free_index = (int*)malloc(size * sizeof(int));
int* visited = (int*) malloc(size * sizeof(int));
for(size_t i = 0; i < size; i++) {
    free_index[i] = 0;
    visited[i] = 0;
}
```

Explanation:

- `continous_ranks` stores the number of continuous ranks to consider for communication, calculated as $D2 - D1$.
- `jump` indicates how much to increment when skipping processed ranks.
- `processes_states_flow` is a 2D array of `pInfo` structures, storing communication info for each process.
- `free_index[i]` keeps track of the next free slot in `processes_states_flow[i]`.
- `visited[i]` marks whether a process `i` has been processed while constructing the communication pattern.

1.2.4 Main Logic Defining Communication Pattern

Listing 4: Communication Pattern Generation Logic

```
for(int p1 = 0; p1 < size; p1+=continous_ranks) {
    for(int p2 = p1, conrun = 0; p2 < size; conrun = (conrun + 1) % continous_ranks
        ) {
        if(visited[p2] == 1) {
            p2 += ((conrun + 1) == continous_ranks ? jump : 1);
            continue;
        }
    }
}
```

```

visited[p2] = 1;

if(p2 + D1 < size && p2 + D2 < size) {
    // for sender
    processes_states_flow[p2][free_index[p2]++] = (struct pInfo) {true, p2
        + D1, true};
    processes_states_flow[p2][free_index[p2]++] = (struct pInfo) {true, p2
        + D2, false};
    processes_states_flow[p2][free_index[p2]++] = (struct pInfo) {false, p2
        + D1, true};
    processes_states_flow[p2][free_index[p2]++] = (struct pInfo) {false, p2
        + D2, false};

    // for receiver
    processes_states_flow[p2 + D1][free_index[p2 + D1]++] = (struct pInfo)
        {false, p2, true};
    processes_states_flow[p2 + D1][free_index[p2 + D1]++] = (struct pInfo)
        {true, p2, true};
    processes_states_flow[p2 + D2][free_index[p2 + D2]++] = (struct pInfo)
        {false, p2, false};
    processes_states_flow[p2 + D2][free_index[p2 + D2]++] = (struct pInfo)
        {true, p2, false};

    valid_sender |= (rank == p2 ? 1 : 0);
    valid_senders_count += 1;
}
else if(p2 + D1 < size) {
    processes_states_flow[p2][free_index[p2]++] = (struct pInfo) {true, p2
        + D1, true};
    processes_states_flow[p2][free_index[p2]++] = (struct pInfo) {false, p2
        + D1, true};
    processes_states_flow[p2 + D1][free_index[p2 + D1]++] = (struct pInfo)
        {false, p2, true};
    processes_states_flow[p2 + D1][free_index[p2 + D1]++] = (struct pInfo)
        {true, p2, true};

    valid_sender |= (rank == p2 ? 1 : 0);
    valid_senders_count += 1;
}

p2 += ((conrun + 1) == continous_ranks ? jump : 1);
}
}

```

Explanation:

The communication pattern is generated using a nested loop over all processes. Each process determines whom to send data to and from whom to receive data at distances D1 and D2. We maintain a `visited[]` array to avoid redundant processing.

- The outer loop iterates over process blocks of size `continous_ranks = D2 - D1`.
- The inner loop iterates through each process in the current block using index `p2`.
- For each process, if both D1 and D2 targets exist, it adds four entries for sending and four for receiving in `processes_states_flow`.
- If only D1 exists, two entries are added for sending and two for receiving.

- The `jump` variable moves to the next unprocessed block efficiently.
- We set `valid_sender` flag to 1 for valid senders and also keep track of total number of valid senders which will be used to identify from how many processes, rank 0 has to receive data in the end.

1.3 Making Send/Receive Calls

Listing 5: Sending Part

```
for(size_t iter = 1; iter <= T; iter++) {
    for(size_t i = 0; i < MAX_COM; i++) {
        struct pInfo p = processes_states_flow[rank][i];
        if(p.comm_rank == -1) break;

        if(p.isSend == true) {
            if(p.isatD1orD2 == true) {
                if(p.comm_rank > rank) { // sending to rank at D1 distance far ahead
                    MPI_Send(data_for_sending_to_D1, M, MPI_DOUBLE, p.comm_rank, p.
                        comm_rank, MPI_COMM_WORLD);
                }
                else { // sending to rank at D1 distance far behind
                    prepare_data_at_D1(data_received, M);
                    MPI_Send(data_received, M, MPI_DOUBLE, p.comm_rank, p.comm_rank,
                        MPI_COMM_WORLD);
                }
            }
            else {
                if(p.comm_rank > rank) { // sending to rank at D2 distance far ahead
                    MPI_Send(data_for_sending_to_D2, M, MPI_DOUBLE, p.comm_rank, p.
                        comm_rank, MPI_COMM_WORLD);
                }
                else { // sending to rank at D2 distance far behind
                    prepare_data_at_D2(data_received, M);
                    MPI_Send(data_received, M, MPI_DOUBLE, p.comm_rank, p.comm_rank,
                        MPI_COMM_WORLD);
                }
            }
        }
    }
}
```

The above code handles the send part of the communication. Different conditions are handled in different blocks(forward send / backward send/ send to D1/ send to D2).

Listing 6: Receiving Part

```
else {
    if(p.isatD1orD2 == true) {
        if(p.comm_rank < rank) { // receiving from rank at D1 distance far behind
            MPI_Recv(data_received, M, MPI_DOUBLE, p.comm_rank, rank,
                MPI_COMM_WORLD, &status);
        }
        else { // receiving from rank at D1 distance far ahead
            MPI_Recv(data_received, M, MPI_DOUBLE, p.comm_rank, rank,
                MPI_COMM_WORLD, &status);
        }
    }
}
```

```

        if(iter == T) max_of_D1 = fmax(max_of_D1, find_max_data(
            data_received, M));
        else prepare_data_for_D1(data_for_sending_to_D1, data_received,
            M);
    }
}
else {
    if(p.comm_rank < rank) { // receiving from rank at D2 distance far behind
        MPI_Recv(data_received, M, MPI_DOUBLE, p.comm_rank, rank,
            MPI_COMM_WORLD, &status);
    }
    else { // receiving from rank at D2 distance far ahead
        MPI_Recv(data_received, M, MPI_DOUBLE, p.comm_rank, rank,
            MPI_COMM_WORLD, &status);

        if(iter == T) max_of_D2 = fmax(max_of_D2, find_max_data(
            data_received, M));
        else prepare_data_for_D2(data_for_sending_to_D2, data_received,
            M);
    }
}
}
}
}
}

```

Here, processes are collecting the data from processes D1 and D2 ahead or behind. Max at D1 and D2 is updated at the end of last iteration.

1.4 Gathering max of D1 and D2 from valid senders at rank 0

```

// valid senders except rank 0 sending max of data_recieved_D1 and max of
data_recieved_D2 to rank 0
if(rank != 0 && valid_sender == 1) {
    double sbuf[2] = {max_of_D1, max_of_D2};
    MPI_Send(sbuf, 2, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);
}

// merging logic (valid senders sending computed_data_recieved to rank 0)
if(rank == 0) {
    int curr_valid_senders_count = 1;
    double recvbuf[2];

    while(curr_valid_senders_count < valid_senders_count) {
        MPI_Recv(recvbuf, 2, MPI_DOUBLE, MPI_ANY_SOURCE, MPI_ANY_TAG,
            MPI_COMM_WORLD, &status);

        max_of_D1 = fmax(max_of_D1, recvbuf[0]);
        max_of_D2 = fmax(max_of_D2, recvbuf[1]);
        curr_valid_senders_count += 1;
    }
}
}

```

Explanation:

Here, all valid sender ranks except 0 are sending its max of D1 and D2 to rank 0. Rank 0 receives them and keeps updating max of D1 and D2 locally and finally prints them along with the maximum time taken by the P processes.

1.5 Handling log of negative values occurred while processing

```
void prepare_data_at_D2(double* data_received, const int M) {  
    const double eps = DBL_MIN, logeps = log(eps);  
    for(size_t i = 0; i < M; i++) {  
        data_received[i] = (data_received[i] <= 0 ? logeps : log(data_received[i]));  
    }  
}
```

In order to deal with this, we are taking log of minimum possible double value (ie 2.2250738585072014e-308) and storing it, inplace of taking log of actual received element for correct efficient working of the logic.

2 Code Compilation and Execution Instructions

1. Navigate to the assignment directory:

```
cd Group22
```

2. Compile the src.c file using:

```
mpicc -o src src.c -lm
```

This will generate the executable file within the same directory named **src**

3. Execute the programs as follows:

- For $P = 8$:

```
sbtach job_scripts/run_8.sh
```

- For $P = 16$:

```
sbtach job_scripts/run_16.sh
```

- For $P = 32$:

```
sbtach job_scripts/run_32.sh
```

On executing any command, it will create a `mpi_<job_id>.out` and `mpi_<job_id>.err` file within `job_output_files` and it will append `<run_no, P, M, D1, D2, T, Seed, MaxD1, MaxD2, Time>` info into `timing.txt` file which will be used to generate boxplot.

4. Compile and Execute `plot_script.c` locally, keeping `timing.txt` in the same folder as follows:

```
gcc plot_script.c -o plot_script  
./plot_script
```

This will generate the `boxplot_time_vs_process.png`

3 Results

Figure 1 shows the execution time (in seconds) versus the number of processes (P) for multiple runs and data sizes. The y-axis represents the execution time, while the x-axis represents the number of processes used. The figure is plotted as a boxplot to show the distribution of execution times across different runs for each process count.

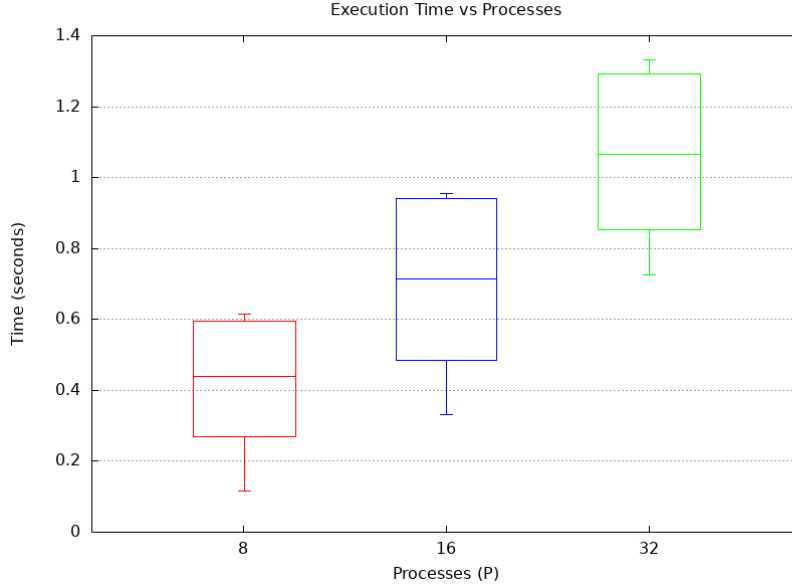


Figure 1: Execution time vs number of processes.

From the figure 1 and Table 1, we observe the following:

- For $P = 8$, the execution times are relatively low, ranging between approximately 0.117 s and 0.613 s. The median execution time is around 0.42 s, indicating low variability across different runs.
- For $P = 16$, the execution times increase, ranging from around 0.330 s to 0.953 s. The median execution time is approximately 0.71 s. This increase is expected as more processes introduce additional communication overhead.
- For $P = 32$, the execution times further increase, ranging from around 0.72 s to 1.33 s, with a median execution time of about 1.08 s. The spread of execution times also becomes wider, reflecting higher variability due to increased process communication and synchronization.
- Overall, the execution time grows with the number of processes. This is primarily due to the overhead of MPI communication, especially when processes exchange data at distances D1 and D2.

Table 1: Execution times for different runs, process counts, and data sizes.

Run	P	M	Time (s)
P = 8			
1	8	262144	0.11700
1	8	1048576	0.59655
2	8	262144	0.26662
2	8	1048576	0.60020
3	8	262144	0.26872
3	8	1048576	0.59208
4	8	262144	0.28175
4	8	1048576	0.59388
5	8	262144	0.28601
5	8	1048576	0.61380
P = 16			
1	16	262144	0.33040
1	16	1048576	0.94154
2	16	262144	0.49283
2	16	1048576	0.92131
3	16	262144	0.48349
3	16	1048576	0.95135
4	16	262144	0.50627
4	16	1048576	0.93225
5	16	262144	0.47984
5	16	1048576	0.95384
P = 32			
1	32	262144	0.72580
1	32	1048576	1.27662
2	32	262144	0.85604
2	32	1048576	1.31034
3	32	262144	0.85416
3	32	1048576	1.29138
4	32	262144	0.85321
4	32	1048576	1.28836
5	32	262144	0.77293
5	32	1048576	1.33142

These results show that while parallelization with more processes can potentially speed up computation for very large data, the communication overhead can dominate for smaller data sizes, leading to increased execution times. The boxplots provide a clear visual indication of variability and possible outliers in execution time for each process count.

4 Conclusions

Based on the results shown in Figure 1 and Table 1, we can conclude:

- The execution time increases with the number of processes due to communication overhead, even though more processes can help parallelize computation.
- For smaller process counts ($P = 8$), execution times are low and stable across runs.
- As the process count increases ($P = 16$ and $P = 32$), execution times increase and variability becomes more pronounced due to MPI communication and synchronization delays.
- The algorithm scales reasonably but is sensitive to the distance-based communication pattern (D1 and D2) which introduces extra synchronization costs.
- Optimization opportunities includes using non-blocking MPI calls to improve scalability.